epoll() - High-Performance I/O for Linux

Theory, Edge Cases & Code Examples

- ✓ The Problem with poll()
- ✓ What is epoll()?
- ✓ How epoll() Works Internally
- ✓ epoll API in Detail
- ✓ Level-Triggered vs Edge-Triggered

→ Swipe to learn →

epoll() - High-Performance I/O for Linux

Theory

📌 The Problem with poll()

In Topic 4, we learned poll() solves select()'s FD_SETSIZE limitation. However, poll() still has O(n) complexity:

poll()'s limitations:

```
// With 10,000 connections
std::vector<struct pollfd> fds(10000);

// ❌ Kernel must check ALL 10,000 FDs every call
int ready = poll(fds.data(), fds.size(), -1);

// ❌ Userspace must iterate ALL 10,000 FDs
for (auto& pfd : fds) {
    if (pfd.revents & POLLIN) {
        handle_read(pfd.fd);
    }
}

// Total: O(10000) in kernel + O(10000) in userspace = O(20000) operations
```

Real-world impact:
- 10,000 connections, 100 active: poll() checks all 10,000 (99% wasted)
- CPU usage scales linearly with total FDs, not active FDs
- High latency under load

---

epoll() - High-Performance I/O for Linux

Theory

📌 What is epoll()?

`epoll()` is a Linux-specific I/O event notification mechanism (since Linux 2.5.44, 2002) that provides **O(1) event registration** and **O(num_active) event retrieval**.

Key innovation: Instead of passing the entire FD set every time, you **register FDs once** and the kernel maintains an internal data structure. Only **ready FDs** are returned.

Function signatures:

```
#include <sys/epoll.h>

// Create epoll instance
int epoll_create1(int flags);

// Control operations (add, modify, delete)
int epoll_ctl(int epfd, int op, int fd, struct epoll_event *event);

// Wait for events
int epoll_wait(int epfd, struct epoll_event *events,
               int maxevents, int timeout);
```

Comparison:

Operation	poll()	epoll()
Setup	Create array	`epoll_create1(0)`
Add FD	`push_back()` (O(1))	`epoll_ctl(ADD)` (O(log n))
Remove FD	`erase()` (O(n))	`epoll_ctl(DEL)` (O(log n))
Wait	`poll()` checks all (O(n))	`epoll_wait()` returns ready (O(num_ready))
Total complexity	O(n) every wait	O(num_ready)

---

epoll() - High-Performance I/O for Linux

Theory

📌 How epoll() Works Internally

Data structures:

```
┌──────────────────────────────────────────┐
│         epoll Instance (epfd)            │
├──────────────────────────────────────────┤
│  Red-Black Tree (Interest List)          │
│   ┌──────────────────────────────┐       │
│   │  FD 4: EPOLLIN               │       │
│   │  FD 5: EPOLLIN | EPOLLOUT    │       │
│   │  FD 8: EPOLLIN               │       │
│   │  ... (all registered FDs)    │       │
│   └──────────────────────────────┘       │
│                                          │
│  Ready List (Linked List)                │
│   ┌──────────────────────────────┐       │
│   │  FD 5: EPOLLIN | EPOLLOUT    │       │
│   │  FD 8: EPOLLIN               │       │
│   │  ... (only ready FDs)        │       │
│   └──────────────────────────────┘       │
└──────────────────────────────────────────┘
```

How it works:
1. **epoll_create1()**: Kernel allocates red-black tree + ready list
2. **epoll_ctl(ADD)**: Insert FD into tree (O(log n))
3. **When data arrives**: Kernel moves FD from tree to ready list (O(1))
4. **epoll_wait()**: Return FDs from ready list (O(num_ready))

Why it's fast:
- Kernel maintains state (no copying entire FD set)
- Only ready FDs returned (no scanning inactive FDs)
- O(1) notification when FD becomes ready

---

epoll() - High-Performance I/O for Linux

Theory

📌 epoll API in Detail (1/3)

1. epoll_create1() - Create Instance

```
int epfd = epoll_create1(0);

// flags:
//   0: Default behavior
//   EPOLL_CLOEXEC: Close-on-exec
```

Returns: epoll file descriptor (just like a socket FD)

2. epoll_ctl() - Control Operations

```
int epoll_ctl(int epfd, int op, int fd, struct epoll_event *event);

// op (operation):
//   EPOLL_CTL_ADD: Add FD to interest list
//   EPOLL_CTL_MOD: Modify existing FD's events
//   EPOLL_CTL_DEL: Remove FD from interest list

// event: What events to monitor
struct epoll_event {
    uint32_t events;    // Event flags (EPOLLIN, EPOLLOUT, etc.)
    epoll_data_t data;  // User data (usually stores FD)
};

typedef union epoll_data {
    void *ptr;
    int fd;
    uint32_t u32;
    uint64_t u64;
} epoll_data_t;
```

Adding a FD:

```
struct epoll_event ev;
ev.events = EPOLLIN;        // Monitor reads
ev.data.fd = client_fd;     // Store FD for retrieval

epoll_ctl(epfd, EPOLL_CTL_ADD, client_fd, &ev);
```

epoll() - High-Performance I/O for Linux

Theory

📌 epoll API in Detail (2/3)

Modifying events:

```
ev.events = EPOLLIN | EPOLLOUT;  // Now also monitor writes
epoll_ctl(epfd, EPOLL_CTL_MOD, client_fd, &ev);
```

Removing a FD:

```
epoll_ctl(epfd, EPOLL_CTL_DEL, client_fd, NULL);  // event can be NULL
```

3. epoll_wait() - Wait for Events

```
struct epoll_event events[MAX_EVENTS];

int nready = epoll_wait(epfd, events, MAX_EVENTS, timeout_ms);

// timeout_ms:
//   -1: Block forever
//   0: Non-blocking (return immediately)
//   >0: Timeout in milliseconds

// Returns:
//   >0: Number of ready FDs (filled in events array)
//   0: Timeout
//   -1: Error
```

Processing events:

epoll() - High-Performance I/O for Linux

Theory

📌 epoll API in Detail (3/3)

```
for (int i = 0; i < nready; i++) {
    int fd = events[i].data.fd;

    if (events[i].events & EPOLLIN) {
        // Read ready
        handle_read(fd);
    }

    if (events[i].events & EPOLLOUT) {
        // Write ready
        handle_write(fd);
    }
}
```

epoll() - High-Performance I/O for Linux

Theory

📌 Level-Triggered vs Edge-Triggered (1/3)

epoll() supports **two notification modes** - the most important concept to understand.

Level-Triggered (default):

```
// Behavior: "FD is ready" (same as poll/select)

ev.events = EPOLLIN;  // Default is level-triggered
epoll_ctl(epfd, EPOLL_CTL_ADD, fd, &ev);

// 1000 bytes arrive
epoll_wait(...);  // Returns: fd ready

recv(fd, buf, 100, 0);  // Read 100 bytes (900 remain)

epoll_wait(...);  // ✅ Returns again (still readable)

recv(fd, buf, 900, 0);  // Read remaining 900 bytes

epoll_wait(...);  // Blocks (no more data)
```

Edge-Triggered (EPOLLET flag):

```
// Behavior: "FD became ready" (only notified on state change)

ev.events = EPOLLIN | EPOLLET;  // Enable edge-triggered
epoll_ctl(epfd, EPOLL_CTL_ADD, fd, &ev);

// 1000 bytes arrive
epoll_wait(...);  // Returns: fd ready

recv(fd, buf, 100, 0);  // Read 100 bytes (900 remain)

epoll_wait(...);  // ❌ Blocks! No new data arrived

// Data is lost unless you read until EAGAIN
```

Edge-triggered requirements:

epoll() - High-Performance I/O for Linux

Theory

📌 Level-Triggered vs Edge-Triggered (2/3)

```
// 1. Socket MUST be non-blocking
int flags = fcntl(fd, F_GETFL, 0);
fcntl(fd, F_SETFL, flags | O_NONBLOCK);

// 2. MUST read until EAGAIN
while (true) {
    int n = recv(fd, buf, sizeof(buf), 0);

    if (n < 0) {
        if (errno == EAGAIN || errno == EWOULDBLOCK) {
            break;  // ✅ No more data
        }
        // Error
    }

    if (n == 0) {
        // EOF
        break;
    }

    process_data(buf, n);
}
```

Comparison:

epoll() - High-Performance I/O for Linux

Theory

📌 Level-Triggered vs Edge-Triggered (3/3)

Aspect	Level-Triggered	Edge-Triggered
Notification	"Is ready"	"Became ready"
Repeat	Every epoll_wait()	Only on state change
Partial read	Still notified	NOT notified
Complexity	Simpler	More complex
Performance	Slightly slower	Slightly faster
Use case	Most applications	High-performance servers

When to use edge-triggered:
- ✅ Extremely high connection count (>100k)
- ✅ Can ensure complete reads/writes
- ✅ Want to avoid wakeup storms (thundering herd)
- ❌ Easier to make mistakes (data loss)

Recommendation: Start with level-triggered. Only use edge-triggered after profiling shows it's needed.

---

epoll() - High-Performance I/O for Linux

Theory

📌 Event Flags (1/2)

Input flags (set in events field):

```
EPOLLIN        // Data available for reading
EPOLLOUT       // Can write without blocking
EPOLLPRI       // High-priority data (OOB)
EPOLLERR       // Error condition (always monitored)
EPOLLHUP       // Hang up (always monitored)
EPOLLET        // Edge-triggered mode
EPOLLONESHOT   // One-shot mode (disable after event)
EPOLLRDHUP     // Peer shutdown write half (since Linux 2.6.17)
```

EPOLLET - Edge-Triggered:

```
ev.events = EPOLLIN | EPOLLET;
```

EPOLLONESHOT - One-Shot Mode:

```
// Used with thread pools: Disable FD after event until re-armed

ev.events = EPOLLIN | EPOLLONESHOT;
epoll_ctl(epfd, EPOLL_CTL_ADD, fd, &ev);

// After event
epoll_wait(...);  // Returns fd

// FD now disabled (won't get more events)
// Worker thread processes FD

// Re-arm after processing
ev.events = EPOLLIN | EPOLLONESHOT;
epoll_ctl(epfd, EPOLL_CTL_MOD, fd, &ev);
```

EPOLLRDHUP - Peer Shutdown:

```
// Detect half-closed connections (peer called shutdown(SHUT_WR))

ev.events = EPOLLIN | EPOLLRDHUP;

if (events[i].events & EPOLLRDHUP) {
// ...continues on next slide
```

📌 Event Flags (2/2)

[Code continues across slides]

```
// ...continued from previous slide
    // Peer shutdown write side (but might still be reading)
}
```

epoll() - High-Performance I/O for Linux

Theory

📌 Performance Characteristics (1/2)

Time complexity:

Operation	poll()	epoll()
Add FD	O(1)	O(log n)
Remove FD	O(n)	O(log n)
Wait	O(n)	O(num_ready)
Modify events	O(1)	O(log n)

Real-world performance:

```
Scenario: 10,000 connections, 100 active

poll():
- poll() call: 10ms (check all 10k FDs)
- Iterate results: 10ms (scan all 10k)
- Total: 20ms per iteration

epoll():
- epoll_wait() call: 0.2ms (return 100 ready)
- Process results: 0.1ms (process 100)
- Total: 0.3ms per iteration

Speedup: 66x faster!
```

Scalability:

epoll() - High-Performance I/O for Linux

Theory

📌 Performance Characteristics (2/2)

Connections	Active	poll() latency	epoll() latency	Speedup
1,000	100	2 ms	0.3 ms	6.7x
10,000	100	20 ms	0.3 ms	66x
100,000	1,000	200 ms	3 ms	66x
1,000,000	10,000	2,000 ms	30 ms	66x

Key insight: epoll() performance scales with **active** connections, not total connections.

epoll() - High-Performance I/O for Linux

Theory

📌 When to Use epoll()

Use epoll() when:

✅ **>1000 concurrent connections**

```
// epoll() shines with high connection counts
```

✅ **Most connections idle** (<1% active)

```
// Long-polling, webhooks, monitoring
```

✅ **Linux-only deployment**

```
// epoll() is Linux-specific
```

✅ **Need maximum performance**

```
// Web servers, proxies, load balancers
```

✅ **C10K problem** (10,000+ concurrent connections)

Don't use epoll() when:

❌ **<1000 connections**

```
// poll() overhead negligible
```

❌ **Need portability** (Windows, BSD, macOS)

```
// Use poll() or abstraction library (libevent, Boost.Asio)
```

❌ **Most connections active** (>50%)

```
// epoll() advantage minimal
```

❌ **Simplicity preferred over performance**

```
// poll() easier to get right
```

---

epoll() - High-Performance I/O for Linux

Theory

📌 Limitations and Gotchas

1. Linux-only:

```
// epoll() not available on:
// - Windows (use IOCP)
// - BSD/macOS (use kqueue)
// - Solaris (use /dev/poll)
```

2. Regular files don't work:

```
int file_fd = open("file.txt", O_RDONLY);

// ❌ Always returns ready (can't epoll regular files)
epoll_ctl(epfd, EPOLL_CTL_ADD, file_fd, &ev);
epoll_wait(...);  // Returns immediately
```

3. Edge-triggered complexity:

```
// Easy to lose data if not careful
// Must read until EAGAIN
// Must use non-blocking sockets
```

4. EPOLLONESHOT re-arming overhead:

```
// epoll_ctl(MOD) is O(log n)
// Can be expensive with many threads
```

5. FD reuse after close():

```
// If FD number reused, old registrations may interfere
close(fd);  // FD 4 closed

int new_fd = accept(...);  // new_fd might be 4 again
// Old epoll registration for FD 4 still exists!

// Solution: Always epoll_ctl(DEL) before close()
epoll_ctl(epfd, EPOLL_CTL_DEL, fd, NULL);
close(fd);
```

---

epoll() - High-Performance I/O for Linux

Edge Cases

📌 1. Edge-Triggered EAGAIN Handling (1/2)

Problem: Forgetting to read until EAGAIN loses data in edge-triggered mode.

```
// ❌ WRONG: Only one recv() call
ev.events = EPOLLIN | EPOLLET;
epoll_ctl(epfd, EPOLL_CTL_ADD, fd, &ev);

if (events[i].events & EPOLLIN) {
    char buf[1024];
    recv(fd, buf, sizeof(buf), 0);  // ❌ If 2000 bytes arrived, 1000 lost!
}
```

Why it's wrong:

1. 2000 bytes arrive
2. epoll_wait() returns fd
3. recv() reads 1024 bytes
4. 976 bytes remain in buffer
5. epoll_wait() won't return again (edge-triggered)
6. **976 bytes lost forever**

Solution: Read until EAGAIN

```
// ✅ Correct: Read loop until EAGAIN
if (events[i].events & EPOLLIN) {
    while (true) {
        char buf[1024];
        int n = recv(fd, buf, sizeof(buf), 0);

        if (n < 0) {
            if (errno == EAGAIN || errno == EWOULDBLOCK) {
                break;  // ✅ No more data
            }

            // Real error
            perror("recv");
            close(fd);
            break;
        }
// ...continues on next slide
```

epoll() - High-Performance I/O for Linux

Edge Cases

📌 1. Edge-Triggered EAGAIN Handling (2/2)

[Code continues across slides]

```
// ...continued from previous slide

        if (n == 0) {
            // EOF
            close(fd);
            break;
        }

        process_data(buf, n);
    }
}
```

Testing for this bug:

```
# Send large data to trigger partial read
dd if=/dev/zero bs=1M count=10 | nc localhost 8080

# Check server received all data (not just first recv)
```

epoll() - High-Performance I/O for Linux

Edge Cases

📌 2. EPOLLONESHOT Re-arming Race (1/2)

Problem: Not re-arming EPOLLONESHOT FD allows events to be lost.

```
// ❌ Worker thread doesn't re-arm
void worker_thread() {
    while (true) {
        int fd = work_queue.pop();

        // Process FD
        handle_client(fd);

        // ❌ Forgot to re-arm! FD now permanently disabled
    }
}
```

Why it happens:

1. Main thread: epoll_wait() returns fd
2. Main thread: Adds fd to work queue
3. FD now disabled (EPOLLONESHOT)
4. Worker processes fd
5. **FD never re-armed - future events lost**

Solution: Always re-arm after processing

```
// ✅ Correct: Re-arm in worker
void worker_thread() {
    while (true) {
        int fd = work_queue.pop();

        // Process FD
        handle_client(fd);

        // ✅ Re-arm for next event
        struct epoll_event ev;
        ev.events = EPOLLIN | EPOLLONESHOT;
        ev.data.fd = fd;
        epoll_ctl(epfd, EPOLL_CTL_MOD, fd, &ev);
    }
}
```

📌 2. EPOLLONESHOT Re-arming Race (2/2)

Alternative: Use separate epoll instance per thread (no EPOLLONESHOT needed).

epoll() - High-Performance I/O for Linux

Edge Cases

📌 3. FD Reuse After close()

Problem: FD numbers are reused by kernel, causing epoll confusion.

```
// ❌ Dangerous sequence
int fd = accept(...);  // FD 5 accepted

epoll_ctl(epfd, EPOLL_CTL_ADD, 5, &ev);  // Register FD 5

close(5);  // Close FD 5 (but still registered in epoll!)

// Later...
int new_fd = accept(...);  // new_fd might be 5 again!

// epoll still thinks FD 5 is the old connection!
// Events for new connection go to old handlers
```

Why it's dangerous:
- Kernel reuses lowest available FD number
- epoll registrations survive close()
- New FD gets events meant for old FD

Solution: Always DEL before close()

```
// ✅ Correct order
epoll_ctl(epfd, EPOLL_CTL_DEL, fd, NULL);  // ✅ Remove from epoll first
close(fd);                                  // ✅ Then close
```

Kernel behavior (since Linux 2.6.9):
- close() automatically removes FD from epoll in recent kernels
- But explicit DEL is clearer and works on all versions

---

epoll() - High-Performance I/O for Linux

Edge Cases

📌 4. Large maxevents vs Small maxevents (1/2)

Problem: Choosing wrong maxevents causes performance issues.

```
// ❌ maxevents too small - starvation
struct epoll_event events[10];
int nready = epoll_wait(epfd, events, 10, -1);  // Only get 10 events

// With 1000 FDs ready, only 10 processed per iteration
// Other 990 FDs starve
```

Impact:
- maxevents=10, 1000 ready: Need 100 iterations to process all
- High latency for connections at end of queue
- Unfair scheduling

Solution 1: Reasonable maxevents

```
// ✅ Good default
const int MAX_EVENTS = 128;
struct epoll_event events[MAX_EVENTS];
```

Solution 2: Process until empty

```
while (true) {
    int nready = epoll_wait(epfd, events, MAX_EVENTS, 0);  // Non-blocking

    if (nready <= 0) break;  // No more ready FDs

    for (int i = 0; i < nready; i++) {
        handle_event(events[i]);
    }
}
```

Opposite problem: maxevents too large

```
// ❌ Wastes memory
struct epoll_event events[100000];  // 800 KB on stack!

// Only 100 FDs typically ready at once
```

📌 4. Large maxevents vs Small maxevents (2/2)

Recommendation: maxevents = 128 to 1024 (good balance).

---

epoll() - High-Performance I/O for Linux

Edge Cases

📌 5. Level-Triggered Starvation (1/2)

Problem: One FD always ready prevents others from being processed.

```
// ❌ Level-triggered busy loop
while (true) {
    int nready = epoll_wait(epfd, events, 128, -1);

    for (int i = 0; i < nready; i++) {
        int fd = events[i].data.fd;

        if (events[i].events & EPOLLIN) {
            char buf[1024];
            int n = recv(fd, buf, sizeof(buf), 0);
            // Only read 1024 bytes, not all data
        }
    }
}

// If FD has 1 MB queued:
// - epoll_wait() returns FD every iteration
// - Other FDs starve
```

Why it happens:
- Level-triggered: FD reported while data remains
- If you don't read all data, FD stays ready
- epoll_wait() keeps returning same FD

Solution 1: Read until would block

```
// ✅ Read all data
while (true) {
    int n = recv(fd, buf, sizeof(buf), MSG_DONTWAIT);
    if (n <= 0) break;
    process(buf, n);
}
```

Solution 2: Use edge-triggered

```
// Edge-triggered forces complete read
ev.events = EPOLLIN | EPOLLET;
```

📌 5. Level-Triggered Starvation (2/2)

Solution 3: Round-robin fairness

```
// Process each FD's first packet, then next iteration
for (int i = 0; i < nready; i++) {
    int n = recv(fd, buf, sizeof(buf), MSG_DONTWAIT);
    if (n > 0) {
        process(buf, n);
        // Don't loop - give other FDs a chance
    }
}
```

epoll() - High-Performance I/O for Linux

Code Examples

📌 1. Basic Echo Server with epoll (Level-Triggered) (1/4)

Purpose: Minimal echo server using epoll in level-triggered mode.

Concepts: epoll_create1, epoll_ctl, epoll_wait, level-triggered behavior.

Compilation and Testing:

Key points:
- Level-triggered (default)
- Non-blocking sockets
- Single recv() per event (level-triggered allows this)

```
#include <iostream>
#include <sys/epoll.h>
#include <sys/socket.h>
#include <netinet/in.h>
#include <unistd.h>
#include <cstring>
#include <fcntl.h>

const int PORT = 8080;
const int MAX_EVENTS = 128;
const int BUFFER_SIZE = 1024;

void set_nonblocking(int fd) {
    int flags = fcntl(fd, F_GETFL, 0);
    fcntl(fd, F_SETFL, flags | O_NONBLOCK);
}

int main() {
    // Create server socket
    int server_fd = socket(AF_INET, SOCK_STREAM, 0);
    if (server_fd < 0) {
        perror("socket");
        return 1;
    }

    int opt = 1;
    setsockopt(server_fd, SOL_SOCKET, SO_REUSEADDR, &opt, sizeof(opt));

// ...continues on next slide
```

epoll() - High-Performance I/O for Linux

Code Examples

📌 1. Basic Echo Server with epoll (Level-Triggered) (2/4)

```
// ...continued from previous slide
    struct sockaddr_in server_addr;
    server_addr.sin_family = AF_INET;
    server_addr.sin_addr.s_addr = INADDR_ANY;
    server_addr.sin_port = htons(PORT);

    if (bind(server_fd, (struct sockaddr*)&server_addr, sizeof(server_addr)) < 0) {
        perror("bind");
        close(server_fd);
        return 1;
    }

    if (listen(server_fd, 128) < 0) {
        perror("listen");
        close(server_fd);
        return 1;
    }

    set_nonblocking(server_fd);

    std::cout << "Echo server listening on port " << PORT << " (epoll level-triggered)\n";

    // Create epoll instance
    int epfd = epoll_create1(0);
    if (epfd < 0) {
        perror("epoll_create1");
        close(server_fd);
        return 1;
    }

    // Add server socket to epoll
    struct epoll_event ev, events[MAX_EVENTS];
    ev.events = EPOLLIN;
    ev.data.fd = server_fd;

    if (epoll_ctl(epfd, EPOLL_CTL_ADD, server_fd, &ev) < 0) {
        perror("epoll_ctl");
        close(server_fd);
        close(epfd);
        return 1;
    }
// ...continues on next slide
```

epoll() - High-Performance I/O for Linux

Code Examples

📌 1. Basic Echo Server with epoll (Level-Triggered) (3/4)

```
// ...continued from previous slide

    // Event loop
    while (true) {
        int nready = epoll_wait(epfd, events, MAX_EVENTS, -1);

        if (nready < 0) {
            perror("epoll_wait");
            break;
        }

        for (int i = 0; i < nready; i++) {
            int fd = events[i].data.fd;

            if (fd == server_fd) {
                // New connection
                struct sockaddr_in client_addr;
                socklen_t addr_len = sizeof(client_addr);
                int client_fd = accept(server_fd, (struct sockaddr*)&client_addr, &addr_len);

                if (client_fd < 0) {
                    perror("accept");
                    continue;
                }

                set_nonblocking(client_fd);

                // Add to epoll
                ev.events = EPOLLIN;
                ev.data.fd = client_fd;
                epoll_ctl(epfd, EPOLL_CTL_ADD, client_fd, &ev);

                std::cout << "Client " << client_fd << " connected\n";
            }
            else {
                // Client data
                if (events[i].events & EPOLLIN) {
                    char buffer[BUFFER_SIZE];
                    int n = recv(fd, buffer, sizeof(buffer), 0);

                    if (n <= 0) {
// ...continues on next slide
```

epoll() - High-Performance I/O for Linux

Code Examples

📌 1. Basic Echo Server with epoll (Level-Triggered) (4/4)

[Code continues across slides]

```
// ...continued from previous slide
                        // Client disconnected or error
                        if (n == 0) {
                            std::cout << "Client " << fd << " disconnected\n";
                        } else {
                            perror("recv");
                        }

                        epoll_ctl(epfd, EPOLL_CTL_DEL, fd, NULL);
                        close(fd);
                    }
                    else {
                        // Echo back
                        buffer[n] = '\0';
                        std::cout << "Client " << fd << ": " << buffer;
                        send(fd, buffer, n, 0);
                    }
                }

                if (events[i].events & (EPOLLHUP | EPOLLERR)) {
                    std::cout << "Client " << fd << " error/hangup\n";
                    epoll_ctl(epfd, EPOLL_CTL_DEL, fd, NULL);
                    close(fd);
                }
            }
        }
    }

    close(epfd);
    close(server_fd);
    return 0;
}
```

epoll() - High-Performance I/O for Linux

Code Examples

📌 2. Edge-Triggered Echo Server (1/4)

Purpose: Demonstrate edge-triggered mode with proper EAGAIN handling.

Concepts: EPOLLET flag, read-until-EAGAIN loop, non-blocking requirements.

Key differences from level-triggered:
- `EPOLLET` flag on all FDs
- Loop until EAGAIN on accept()
- Loop until EAGAIN on recv()
- Must be non-blocking or data loss occurs

Testing:

```
#include <iostream>
#include <sys/epoll.h>
#include <sys/socket.h>
#include <netinet/in.h>
#include <unistd.h>
#include <fcntl.h>
#include <cstring>
#include <errno.h>

const int PORT = 8080;
const int MAX_EVENTS = 128;
const int BUFFER_SIZE = 1024;

void set_nonblocking(int fd) {
    int flags = fcntl(fd, F_GETFL, 0);
    fcntl(fd, F_SETFL, flags | O_NONBLOCK);
}

int main() {
    // Create server (same as Example 1)
    int server_fd = socket(AF_INET, SOCK_STREAM, 0);
    int opt = 1;
    setsockopt(server_fd, SOL_SOCKET, SO_REUSEADDR, &opt, sizeof(opt));

    struct sockaddr_in server_addr;
    server_addr.sin_family = AF_INET;
    server_addr.sin_addr.s_addr = INADDR_ANY;
// ...continues on next slide
```

epoll() - High-Performance I/O for Linux

Code Examples

📌 2. Edge-Triggered Echo Server (2/4)

```
// ...continued from previous slide
    server_addr.sin_port = htons(PORT);

    bind(server_fd, (struct sockaddr*)&server_addr, sizeof(server_addr));
    listen(server_fd, 128);
    set_nonblocking(server_fd);

    std::cout << "Echo server on port " << PORT << " (EDGE-TRIGGERED)\n";

    // Create epoll with edge-triggered
    int epfd = epoll_create1(0);
    struct epoll_event ev, events[MAX_EVENTS];

    // Add server with EPOLLET
    ev.events = EPOLLIN | EPOLLET;
    ev.data.fd = server_fd;
    epoll_ctl(epfd, EPOLL_CTL_ADD, server_fd, &ev);

    while (true) {
        int nready = epoll_wait(epfd, events, MAX_EVENTS, -1);

        for (int i = 0; i < nready; i++) {
            int fd = events[i].data.fd;

            if (fd == server_fd) {
                // Accept ALL pending connections (edge-triggered)
                while (true) {
                    struct sockaddr_in client_addr;
                    socklen_t addr_len = sizeof(client_addr);
                    int client_fd = accept(server_fd, (struct sockaddr*)&client_addr, &addr_len);

                    if (client_fd < 0) {
                        if (errno == EAGAIN || errno == EWOULDBLOCK) {
                            break;  // No more pending connections
                        }
                        perror("accept");
                        break;
                    }

                    set_nonblocking(client_fd);

// ...continues on next slide
```

epoll() - High-Performance I/O for Linux

Code Examples

📌 2. Edge-Triggered Echo Server (3/4)

```
// ...continued from previous slide
                    // Add with edge-triggered
                    ev.events = EPOLLIN | EPOLLET;
                    ev.data.fd = client_fd;
                    epoll_ctl(epfd, EPOLL_CTL_ADD, client_fd, &ev);

                    std::cout << "Client " << client_fd << " connected\n";
                }
            }
            else {
                if (events[i].events & EPOLLIN) {
                    // Read ALL available data (critical for edge-triggered!)
                    while (true) {
                        char buffer[BUFFER_SIZE];
                        int n = recv(fd, buffer, sizeof(buffer), 0);

                        if (n < 0) {
                            if (errno == EAGAIN || errno == EWOULDBLOCK) {
                                break;  // ✅ All data read
                            }

                            // Real error
                            perror("recv");
                            epoll_ctl(epfd, EPOLL_CTL_DEL, fd, NULL);
                            close(fd);
                            break;
                        }

                        if (n == 0) {
                            // EOF
                            std::cout << "Client " << fd << " disconnected\n";
                            epoll_ctl(epfd, EPOLL_CTL_DEL, fd, NULL);
                            close(fd);
                            break;
                        }

                        // Echo back
                        buffer[n] = '\0';
                        std::cout << "Client " << fd << ": " << buffer;
                        send(fd, buffer, n, MSG_NOSIGNAL);
                    }
// ...continues on next slide
```

epoll() - High-Performance I/O for Linux

Code Examples

📌 2. Edge-Triggered Echo Server (4/4)

[Code continues across slides]

```
// ...continued from previous slide
                }

                if (events[i].events & (EPOLLHUP | EPOLLERR)) {
                    epoll_ctl(epfd, EPOLL_CTL_DEL, fd, NULL);
                    close(fd);
                }
            }
        }
    }

    close(epfd);
    close(server_fd);
    return 0;
}
```

epoll() - High-Performance I/O for Linux

Code Examples

📌 3. Chat Server with epoll (1/4)

Purpose: Multi-client chat with broadcast using epoll.

Concepts: Client tracking, broadcasting to multiple FDs, epoll with metadata.

Testing:

```
#include <iostream>
#include <map>
#include <string>
#include <sys/epoll.h>
#include <sys/socket.h>
#include <netinet/in.h>
#include <arpa/inet.h>
#include <unistd.h>
#include <fcntl.h>
#include <cstring>

const int PORT = 8080;
const int MAX_EVENTS = 128;
const int BUFFER_SIZE = 1024;

struct ClientInfo {
    std::string name;
    std::string address;
};

std::map<int, ClientInfo> clients;

void set_nonblocking(int fd) {
    int flags = fcntl(fd, F_GETFL, 0);
    fcntl(fd, F_SETFL, flags | O_NONBLOCK);
}

void broadcast(const std::string& message, int sender_fd, int server_fd) {
    for (auto& pair : clients) {
        int fd = pair.first;
        if (fd != sender_fd && fd != server_fd) {
            send(fd, message.c_str(), message.size(), MSG_NOSIGNAL);
        }
    }
// ...continues on next slide
```

epoll() - High-Performance I/O for Linux

Code Examples

📌 3. Chat Server with epoll (2/4)

```
// ...continued from previous slide
}

int main() {
    int server_fd = socket(AF_INET, SOCK_STREAM, 0);
    int opt = 1;
    setsockopt(server_fd, SOL_SOCKET, SO_REUSEADDR, &opt, sizeof(opt));

    struct sockaddr_in server_addr;
    server_addr.sin_family = AF_INET;
    server_addr.sin_addr.s_addr = INADDR_ANY;
    server_addr.sin_port = htons(PORT);

    bind(server_fd, (struct sockaddr*)&server_addr, sizeof(server_addr));
    listen(server_fd, 128);
    set_nonblocking(server_fd);

    std::cout << "Chat server on port " << PORT << "\n";

    int epfd = epoll_create1(0);
    struct epoll_event ev, events[MAX_EVENTS];

    ev.events = EPOLLIN;
    ev.data.fd = server_fd;
    epoll_ctl(epfd, EPOLL_CTL_ADD, server_fd, &ev);

    while (true) {
        int nready = epoll_wait(epfd, events, MAX_EVENTS, -1);

        for (int i = 0; i < nready; i++) {
            int fd = events[i].data.fd;

            if (fd == server_fd) {
                // New connection
                struct sockaddr_in client_addr;
                socklen_t addr_len = sizeof(client_addr);
                int client_fd = accept(server_fd, (struct sockaddr*)&client_addr, &addr_len);

                if (client_fd >= 0) {
                    set_nonblocking(client_fd);

// ...continues on next slide
```

epoll() - High-Performance I/O for Linux

Code Examples

📌 3. Chat Server with epoll (3/4)

```
// ...continued from previous slide
                    ClientInfo info;
                    info.name = "User" + std::to_string(client_fd);
                    info.address = inet_ntoa(client_addr.sin_addr);
                    clients[client_fd] = info;

                    ev.events = EPOLLIN;
                    ev.data.fd = client_fd;
                    epoll_ctl(epfd, EPOLL_CTL_ADD, client_fd, &ev);

                    std::string welcome = "Welcome! You are " + info.name + "\n";
                    send(client_fd, welcome.c_str(), welcome.size(), 0);

                    std::string join_msg = info.name + " joined the chat\n";
                    std::cout << join_msg;
                    broadcast(join_msg, client_fd, server_fd);
                }
            }
            else {
                if (events[i].events & EPOLLIN) {
                    char buffer[BUFFER_SIZE];
                    int n = recv(fd, buffer, sizeof(buffer), 0);

                    if (n <= 0) {
                        // Disconnected
                        std::string goodbye = clients[fd].name + " left the chat\n";
                        std::cout << goodbye;
                        broadcast(goodbye, fd, server_fd);

                        epoll_ctl(epfd, EPOLL_CTL_DEL, fd, NULL);
                        clients.erase(fd);
                        close(fd);
                    }
                    else {
                        buffer[n] = '\0';
                        std::string message = clients[fd].name + ": " + buffer;
                        std::cout << message;
                        broadcast(message, fd, server_fd);
                    }
                }

// ...continues on next slide
```

epoll() - High-Performance I/O for Linux

Code Examples

📌 3. Chat Server with epoll (4/4)

[Code continues across slides]

```
// ...continued from previous slide
                if (events[i].events & (EPOLLHUP | EPOLLERR)) {
                    if (clients.count(fd)) {
                        std::string goodbye = clients[fd].name + " disconnected\n";
                        std::cout << goodbye;
                        broadcast(goodbye, fd, server_fd);
                        clients.erase(fd);
                    }

                    epoll_ctl(epfd, EPOLL_CTL_DEL, fd, NULL);
                    close(fd);
                }
            }
        }
    }

    close(epfd);
    close(server_fd);
    return 0;
}
```

epoll() - High-Performance I/O for Linux

Code Examples

📌 4. EPOLLOUT for Write Queues (1/5)

Purpose: Handle partial sends with EPOLLOUT and write queues.

Concepts: Dynamic event modification, EPOLLOUT monitoring, queue management.

Key points:
- Only monitor EPOLLOUT when write queue has data
- Use `epoll_ctl(MOD)` to dynamically change events
- Handle partial sends by keeping remainder in queue

🎉 Break 1 Complete!

Topic 5 now has:
- ✅ THEORY_SECTION (comprehensive epoll coverage)
- ✅ 5 EDGE_CASES (EAGAIN handling, EPOLLONESHOT, FD reuse, maxevents, starvation)
- ✅ 4 CODE_EXAMPLES (level-triggered echo, edge-triggered echo, chat server, EPOLLOUT handling)

Remaining for full completion:
- ⏳ 2 more examples (EPOLLONESHOT with thread pool, production-grade server)
- ⏳ Break 2: INTERVIEW_QA (20 questions)
- ⏳ Break 3: PRACTICE_TASKS + QUICK_REFERENCE

Continuing with final 2 examples to complete Break 1...

```
#include <iostream>
#include <map>
#include <queue>
#include <string>
#include <sys/epoll.h>
#include <sys/socket.h>
#include <netinet/in.h>
#include <unistd.h>
#include <fcntl.h>
#include <errno.h>

const int PORT = 8080;
const int MAX_EVENTS = 128;

// ...continues on next slide
```

epoll() - High-Performance I/O for Linux

Code Examples

📌 4. EPOLLOUT for Write Queues (2/5)

```
// ...continued from previous slide
std::map<int, std::queue<std::string>> write_queues;
int epfd;  // Global for update_events

void set_nonblocking(int fd) {
    int flags = fcntl(fd, F_GETFL, 0);
    fcntl(fd, F_SETFL, flags | O_NONBLOCK);
}

void update_events(int fd) {
    struct epoll_event ev;
    ev.data.fd = fd;

    if (write_queues[fd].empty()) {
        // No data to send - only monitor reads
        ev.events = EPOLLIN;
    } else {
        // Data pending - monitor reads AND writes
        ev.events = EPOLLIN | EPOLLOUT;
    }

    epoll_ctl(epfd, EPOLL_CTL_MOD, fd, &ev);
}

void queue_send(int fd, const std::string& data) {
    bool was_empty = write_queues[fd].empty();
    write_queues[fd].push(data);

    if (was_empty) {
        update_events(fd);  // Start monitoring EPOLLOUT
    }
}

int main() {
    int server_fd = socket(AF_INET, SOCK_STREAM, 0);
    int opt = 1;
    setsockopt(server_fd, SOL_SOCKET, SO_REUSEADDR, &opt, sizeof(opt));

    struct sockaddr_in server_addr;
    server_addr.sin_family = AF_INET;
    server_addr.sin_addr.s_addr = INADDR_ANY;
// ...continues on next slide
```

epoll() - High-Performance I/O for Linux

Code Examples

📌 4. EPOLLOUT for Write Queues (3/5)

```
// ...continued from previous slide
    server_addr.sin_port = htons(PORT);

    bind(server_fd, (struct sockaddr*)&server_addr, sizeof(server_addr));
    listen(server_fd, 128);
    set_nonblocking(server_fd);

    std::cout << "Server with EPOLLOUT on port " << PORT << "\n";

    epfd = epoll_create1(0);
    struct epoll_event ev, events[MAX_EVENTS];

    ev.events = EPOLLIN;
    ev.data.fd = server_fd;
    epoll_ctl(epfd, EPOLL_CTL_ADD, server_fd, &ev);

    while (true) {
        int nready = epoll_wait(epfd, events, MAX_EVENTS, -1);

        for (int i = 0; i < nready; i++) {
            int fd = events[i].data.fd;

            if (fd == server_fd) {
                int client_fd = accept(server_fd, NULL, NULL);
                if (client_fd >= 0) {
                    set_nonblocking(client_fd);

                    ev.events = EPOLLIN;
                    ev.data.fd = client_fd;
                    epoll_ctl(epfd, EPOLL_CTL_ADD, client_fd, &ev);

                    write_queues[client_fd] = std::queue<std::string>();
                    std::cout << "Client " << client_fd << " connected\n";
                }
            }
            else {
                if (events[i].events & EPOLLIN) {
                    char buffer[1024];
                    int n = recv(fd, buffer, sizeof(buffer), 0);

                    if (n <= 0) {
// ...continues on next slide
```

epoll() - High-Performance I/O for Linux

Code Examples

📌 4. EPOLLOUT for Write Queues (4/5)

```
// ...continued from previous slide
                        epoll_ctl(epfd, EPOLL_CTL_DEL, fd, NULL);
                        write_queues.erase(fd);
                        close(fd);
                    }
                    else {
                        // Queue large response (simulating heavy load)
                        buffer[n] = '\0';
                        std::string response = "ECHO: " + std::string(buffer);

                        // Multiply by 100 to test partial sends
                        std::string large_response;
                        for (int j = 0; j < 100; j++) {
                            large_response += response;
                        }

                        queue_send(fd, large_response);
                        std::cout << "Queued " << large_response.size()
                                  << " bytes for client " << fd << "\n";
                    }
                }

                if (events[i].events & EPOLLOUT) {
                    if (!write_queues[fd].empty()) {
                        std::string& data = write_queues[fd].front();

                        ssize_t sent = send(fd, data.c_str(), data.size(), MSG_NOSIGNAL);

                        if (sent > 0) {
                            std::cout << "Sent " << sent << " bytes to " << fd << "\n";

                            if (static_cast<size_t>(sent) < data.size()) {
                                // Partial send
                                data.erase(0, sent);
                            } else {
                                // Complete send
                                write_queues[fd].pop();
                            }

                            update_events(fd);  // Update based on queue state
                        }
// ...continues on next slide
```

epoll() - High-Performance I/O for Linux

Code Examples

📌 4. EPOLLOUT for Write Queues (5/5)

[Code continues across slides]

```
// ...continued from previous slide
                        else if (sent < 0 && errno != EAGAIN && errno != EWOULDBLOCK) {
                            epoll_ctl(epfd, EPOLL_CTL_DEL, fd, NULL);
                            write_queues.erase(fd);
                            close(fd);
                        }
                    }
                }

                if (events[i].events & (EPOLLHUP | EPOLLERR)) {
                    epoll_ctl(epfd, EPOLL_CTL_DEL, fd, NULL);
                    write_queues.erase(fd);
                    close(fd);
                }
            }
        }
    }

    close(epfd);
    close(server_fd);
    return 0;
}
```

epoll() - High-Performance I/O for Linux

Code Examples

📌 5. EPOLLONESHOT with Thread Pool (1/6)

Why: EPOLLONESHOT prevents multiple threads from processing the same FD simultaneously. Critical for thread-safe multi-threaded servers.

Problem: With normal epoll, if thread A is processing FD 5 and new data arrives, thread B might also get notified and race with A.

Solution: EPOLLONESHOT disables FD after event delivery. Must re-arm with `epoll_ctl(MOD)` after processing.

Explanation:

1. **EPOLLONESHOT flag:** FD is automatically disabled after event delivery
2. **Thread pool:** 4 worker threads process events from shared queue
3. **Re-arming:** Worker must call `epoll_ctl(MOD)` to re-enable FD after processing
4. **Safety:** No race conditions - only one thread processes FD at a time
5. **Listen socket:** Does NOT use EPOLLONESHOT (multiple threads can accept)

Why EPOLLONESHOT is Critical:

Without EPOLLONESHOT:

With EPOLLONESHOT:

```
#include <sys/epoll.h>
#include <sys/socket.h>
#include <netinet/in.h>
#include <fcntl.h>
#include <unistd.h>
#include <errno.h>
#include <cstring>
#include <iostream>
#include <thread>
#include <vector>
#include <queue>
#include <mutex>
#include <condition_variable>

// Thread-safe task queue
class TaskQueue {
// ...continues on next slide
```

epoll() - High-Performance I/O for Linux

Code Examples

📌 5. EPOLLONESHOT with Thread Pool (2/6)

```
// ...continued from previous slide
private:
    std::queue<std::pair<int, uint32_t>> tasks;  // {fd, events}
    std::mutex mtx;
    std::condition_variable cv;
    bool shutdown = false;

public:
    void push(int fd, uint32_t events) {
        std::lock_guard<std::mutex> lock(mtx);
        tasks.push({fd, events});
        cv.notify_one();
    }

    bool pop(int& fd, uint32_t& events) {
        std::unique_lock<std::mutex> lock(mtx);
        cv.wait(lock, [this] { return !tasks.empty() || shutdown; });

        if (shutdown && tasks.empty()) return false;

        auto task = tasks.front();
        tasks.pop();
        fd = task.first;
        events = task.second;
        return true;
    }

    void stop() {
        {
            std::lock_guard<std::mutex> lock(mtx);
            shutdown = true;
        }
        cv.notify_all();
    }
};

// Worker thread function
void worker(int epfd, TaskQueue& queue) {
    char buf[1024];

    while (true) {
// ...continues on next slide
```

epoll() - High-Performance I/O for Linux

Code Examples

📌 5. EPOLLONESHOT with Thread Pool (3/6)

```
// ...continued from previous slide
        int fd;
        uint32_t events;

        if (!queue.pop(fd, events)) break;  // Shutdown

        // Process the event
        if (events & EPOLLIN) {
            int n = recv(fd, buf, sizeof(buf) - 1, 0);

            if (n > 0) {
                buf[n] = '\0';
                std::cout << "[Thread " << std::this_thread::get_id()
                          << "] FD " << fd << ": " << buf;

                // Echo back
                send(fd, buf, n, 0);

                // ✅ Re-arm EPOLLONESHOT after processing
                struct epoll_event ev;
                ev.events = EPOLLIN | EPOLLONESHOT | EPOLLET;
                ev.data.fd = fd;

                if (epoll_ctl(epfd, EPOLL_CTL_MOD, fd, &ev) < 0) {
                    perror("epoll_ctl MOD");
                }
            } else if (n == 0) {
                std::cout << "[Thread " << std::this_thread::get_id()
                          << "] FD " << fd << " closed\n";
                epoll_ctl(epfd, EPOLL_CTL_DEL, fd, nullptr);
                close(fd);
            } else {
                if (errno != EAGAIN) {
                    perror("recv");
                    epoll_ctl(epfd, EPOLL_CTL_DEL, fd, nullptr);
                    close(fd);
                }
            }
        }
    }
}
// ...continues on next slide
```

epoll() - High-Performance I/O for Linux

Code Examples

📌 5. EPOLLONESHOT with Thread Pool (4/6)

```
// ...continued from previous slide

int main() {
    // Create server socket
    int server_fd = socket(AF_INET, SOCK_STREAM, 0);
    int opt = 1;
    setsockopt(server_fd, SOL_SOCKET, SO_REUSEADDR, &opt, sizeof(opt));

    sockaddr_in addr{};
    addr.sin_family = AF_INET;
    addr.sin_addr.s_addr = INADDR_ANY;
    addr.sin_port = htons(8080);

    bind(server_fd, (sockaddr*)&addr, sizeof(addr));
    listen(server_fd, 128);

    // Create epoll
    int epfd = epoll_create1(0);

    // Add server socket (no ONESHOT needed for listen socket)
    struct epoll_event ev;
    ev.events = EPOLLIN;
    ev.data.fd = server_fd;
    epoll_ctl(epfd, EPOLL_CTL_ADD, server_fd, &ev);

    // Create thread pool
    const int NUM_THREADS = 4;
    TaskQueue queue;
    std::vector<std::thread> workers;

    for (int i = 0; i < NUM_THREADS; i++) {
        workers.emplace_back(worker, epfd, std::ref(queue));
    }

    std::cout << "Thread pool echo server with EPOLLONESHOT on :8080\n";
    std::cout << "Workers: " << NUM_THREADS << "\n";

    struct epoll_event events[128];

    while (true) {
        int nready = epoll_wait(epfd, events, 128, -1);
// ...continues on next slide
```

epoll() - High-Performance I/O for Linux

Code Examples

📌 5. EPOLLONESHOT with Thread Pool (5/6)

[Code continues across slides]

```
// ...continued from previous slide

        for (int i = 0; i < nready; i++) {
            int fd = events[i].data.fd;

            if (fd == server_fd) {
                // Accept new connections
                int client_fd = accept(server_fd, nullptr, nullptr);

                if (client_fd < 0) continue;

                // Set non-blocking
                int flags = fcntl(client_fd, F_GETFL, 0);
                fcntl(client_fd, F_SETFL, flags | O_NONBLOCK);

                // Add with EPOLLONESHOT
                struct epoll_event cev;
                cev.events = EPOLLIN | EPOLLONESHOT | EPOLLET;
                cev.data.fd = client_fd;
                epoll_ctl(epfd, EPOLL_CTL_ADD, client_fd, &cev);

                std::cout << "[Main] New connection: FD " << client_fd << "\n";
            } else {
                // Client event → push to worker queue
                queue.push(fd, events[i].events);
            }
        }
    }

    // Cleanup (unreachable in this example)
    queue.stop();
    for (auto& t : workers) t.join();
    close(epfd);
    close(server_fd);

    return 0;
}
```

epoll() - High-Performance I/O for Linux

Code Examples

📌 5. EPOLLONESHOT with Thread Pool (6/6)

```
['Time | Thread A        | Thread B\n-----|------------------|------------------\nT1   | recv(fd5, ...)   |\nT2   | pr
```

epoll() - High-Performance I/O for Linux

Code Examples

📌 6. Production-Grade epoll Server (1/9)

Why: Real-world servers need connection limits, graceful shutdown, metrics, and error handling. This example shows production patterns.

Production Features:

1. **Connection Limits:** Rejects connections when `MAX_CONNECTIONS` reached
2. **Graceful Shutdown:** SIGINT/SIGTERM closes all connections cleanly
3. **Metrics Tracking:** Bytes sent/received, connection count, errors
4. **TCP_NODELAY:** Disables Nagle algorithm for low latency
5. **Non-blocking Accept:** Loops until EAGAIN in edge-triggered mode
6. **Write Buffering:** Only monitors EPOLLOUT when data pending
7. **Connection State:** Tracks per-connection lifetime and stats
8. **Error Handling:** EPOLLERR, EPOLLHUP, and errno checks
9. **Periodic Reporting:** Prints metrics every 10 seconds

Compile & Test:

Expected Output:

This production server demonstrates all patterns needed for high-performance, reliable network services using epoll().

```
#include <sys/epoll.h>
#include <sys/socket.h>
#include <netinet/in.h>
#include <netinet/tcp.h>
#include <fcntl.h>
#include <unistd.h>
#include <signal.h>
#include <errno.h>
#include <cstring>
#include <iostream>
#include <chrono>
#include <atomic>
#include <unordered_map>

// ...continues on next slide
```

epoll() - High-Performance I/O for Linux

Code Examples

📌 6. Production-Grade epoll Server (2/9)

```
// ...continued from previous slide
// Global state for signal handling
std::atomic<bool> g_running{true};

void signal_handler(int sig) {
    std::cout << "\n[Signal] Received " << sig << ", shutting down gracefully...\n";
    g_running = false;
}

// Per-connection state
struct Connection {
    int fd;
    std::chrono::steady_clock::time_point connected_at;
    size_t bytes_received = 0;
    size_t bytes_sent = 0;
    std::string read_buffer;
    std::string write_buffer;
};

// Server metrics
struct Metrics {
    std::atomic<size_t> total_connections{0};
    std::atomic<size_t> active_connections{0};
    std::atomic<size_t> total_bytes_received{0};
    std::atomic<size_t> total_bytes_sent{0};
    std::atomic<size_t> total_errors{0};

    void print() const {
        std::cout << "\n=== Server Metrics ===\n"
                  << "Total connections: " << total_connections << "\n"
                  << "Active connections: " << active_connections << "\n"
                  << "Total bytes received: " << total_bytes_received << "\n"
                  << "Total bytes sent: " << total_bytes_sent << "\n"
                  << "Total errors: " << total_errors << "\n";
    }
};

int set_nonblocking(int fd) {
    int flags = fcntl(fd, F_GETFL, 0);
    if (flags < 0) return -1;
    return fcntl(fd, F_SETFL, flags | O_NONBLOCK);
// ...continues on next slide
```

epoll() - High-Performance I/O for Linux

Code Examples

📌 6. Production-Grade epoll Server (3/9)

```
// ...continued from previous slide
}

int set_tcp_nodelay(int fd) {
    int flag = 1;
    return setsockopt(fd, IPPROTO_TCP, TCP_NODELAY, &flag, sizeof(flag));
}

void handle_client_read(int epfd, Connection& conn, Metrics& metrics) {
    char buf[4096];

    while (true) {
        int n = recv(conn.fd, buf, sizeof(buf), 0);

        if (n > 0) {
            conn.bytes_received += n;
            metrics.total_bytes_received += n;

            // Echo: Add to write buffer
            conn.write_buffer.append(buf, n);

            // Start monitoring EPOLLOUT
            struct epoll_event ev;
            ev.events = EPOLLIN | EPOLLOUT | EPOLLET;
            ev.data.fd = conn.fd;
            epoll_ctl(epfd, EPOLL_CTL_MOD, conn.fd, &ev);

            break;  // Level-triggered, will get more events if data remains
        } else if (n == 0) {
            // Graceful close
            std::cout << "[FD " << conn.fd << "] Client disconnected\n";
            return;
        } else {
            if (errno == EAGAIN || errno == EWOULDBLOCK) {
                break;  // No more data
            }
            // Error
            perror("recv");
            metrics.total_errors++;
            return;
        }
// ...continues on next slide
```

epoll() - High-Performance I/O for Linux

Code Examples

📌 6. Production-Grade epoll Server (4/9)

```
// ...continued from previous slide
    }
}

void handle_client_write(int epfd, Connection& conn, Metrics& metrics) {
    while (!conn.write_buffer.empty()) {
        int n = send(conn.fd, conn.write_buffer.data(), conn.write_buffer.size(), 0);

        if (n > 0) {
            conn.bytes_sent += n;
            metrics.total_bytes_sent += n;
            conn.write_buffer.erase(0, n);
        } else {
            if (errno == EAGAIN || errno == EWOULDBLOCK) {
                break;  // Socket buffer full, try later
            }
            perror("send");
            metrics.total_errors++;
            return;
        }
    }

    // If write buffer empty, stop monitoring EPOLLOUT
    if (conn.write_buffer.empty()) {
        struct epoll_event ev;
        ev.events = EPOLLIN | EPOLLET;
        ev.data.fd = conn.fd;
        epoll_ctl(epfd, EPOLL_CTL_MOD, conn.fd, &ev);
    }
}

int main() {
    // Setup signal handling
    signal(SIGINT, signal_handler);
    signal(SIGTERM, signal_handler);

    // Configuration
    const int PORT = 8080;
    const int MAX_CONNECTIONS = 10000;
    const int MAX_EVENTS = 128;

// ...continues on next slide
```

epoll() - High-Performance I/O for Linux

Code Examples

📌 6. Production-Grade epoll Server (5/9)

```
// ...continued from previous slide
    // Create server socket
    int server_fd = socket(AF_INET, SOCK_STREAM, 0);
    if (server_fd < 0) {
        perror("socket");
        return 1;
    }

    int opt = 1;
    setsockopt(server_fd, SOL_SOCKET, SO_REUSEADDR, &opt, sizeof(opt));
    set_nonblocking(server_fd);

    sockaddr_in addr{};
    addr.sin_family = AF_INET;
    addr.sin_addr.s_addr = INADDR_ANY;
    addr.sin_port = htons(PORT);

    if (bind(server_fd, (sockaddr*)&addr, sizeof(addr)) < 0) {
        perror("bind");
        close(server_fd);
        return 1;
    }

    if (listen(server_fd, 128) < 0) {
        perror("listen");
        close(server_fd);
        return 1;
    }

    // Create epoll
    int epfd = epoll_create1(0);
    if (epfd < 0) {
        perror("epoll_create1");
        close(server_fd);
        return 1;
    }

    // Add server socket
    struct epoll_event ev;
    ev.events = EPOLLIN | EPOLLET;
    ev.data.fd = server_fd;
// ...continues on next slide
```

epoll() - High-Performance I/O for Linux

Code Examples

📌 6. Production-Grade epoll Server (6/9)

```
// ...continued from previous slide
    epoll_ctl(epfd, EPOLL_CTL_ADD, server_fd, &ev);

    std::unordered_map<int, Connection> connections;
    Metrics metrics;

    std::cout << "=== Production epoll Server ===\n"
              << "Listening on ::" << PORT << "\n"
              << "Max connections: " << MAX_CONNECTIONS << "\n"
              << "Max events per iteration: " << MAX_EVENTS << "\n";

    struct epoll_event events[MAX_EVENTS];
    auto last_metrics_print = std::chrono::steady_clock::now();

    while (g_running) {
        int timeout_ms = 1000;  // 1 second timeout for graceful shutdown
        int nready = epoll_wait(epfd, events, MAX_EVENTS, timeout_ms);

        if (nready < 0) {
            if (errno == EINTR) continue;  // Interrupted by signal
            perror("epoll_wait");
            break;
        }

        for (int i = 0; i < nready; i++) {
            int fd = events[i].data.fd;
            uint32_t ev_flags = events[i].events;

            // Error or hangup
            if (ev_flags & (EPOLLERR | EPOLLHUP)) {
                std::cerr << "[FD " << fd << "] Error/Hangup\n";
                epoll_ctl(epfd, EPOLL_CTL_DEL, fd, nullptr);
                connections.erase(fd);
                metrics.active_connections--;
                close(fd);
                continue;
            }

            // Server socket: Accept new connections
            if (fd == server_fd) {
                while (true) {
// ...continues on next slide
```

epoll() - High-Performance I/O for Linux

Code Examples

📌 6. Production-Grade epoll Server (7/9)

```
// ...continued from previous slide
                    int client_fd = accept(server_fd, nullptr, nullptr);

                    if (client_fd < 0) {
                        if (errno == EAGAIN || errno == EWOULDBLOCK) {
                            break;  // No more connections
                        }
                        perror("accept");
                        metrics.total_errors++;
                        break;
                    }

                    // Connection limit
                    if (metrics.active_connections >= MAX_CONNECTIONS) {
                        std::cerr << "[LIMIT] Max connections reached, rejecting FD "
                                  << client_fd << "\n";
                        close(client_fd);
                        continue;
                    }

                    set_nonblocking(client_fd);
                    set_tcp_nodelay(client_fd);

                    // Add to epoll
                    struct epoll_event cev;
                    cev.events = EPOLLIN | EPOLLET;
                    cev.data.fd = client_fd;
                    epoll_ctl(epfd, EPOLL_CTL_ADD, client_fd, &cev);

                    // Track connection
                    Connection conn;
                    conn.fd = client_fd;
                    conn.connected_at = std::chrono::steady_clock::now();
                    connections[client_fd] = conn;

                    metrics.total_connections++;
                    metrics.active_connections++;

                    std::cout << "[FD " << client_fd << "] New connection (active: "
                              << metrics.active_connections << ")\n";
                }
// ...continues on next slide
```

epoll() - High-Performance I/O for Linux

Code Examples

📌 6. Production-Grade epoll Server (8/9)

```
// ...continued from previous slide
                continue;
            }

            // Client socket: Handle I/O
            auto it = connections.find(fd);
            if (it == connections.end()) continue;

            Connection& conn = it->second;

            if (ev_flags & EPOLLIN) {
                handle_client_read(epfd, conn, metrics);

                // Check if connection should be closed
                if (conn.write_buffer.empty() && conn.bytes_received == 0) {
                    epoll_ctl(epfd, EPOLL_CTL_DEL, fd, nullptr);
                    connections.erase(fd);
                    metrics.active_connections--;
                    close(fd);
                }
            }

            if (ev_flags & EPOLLOUT) {
                handle_client_write(epfd, conn, metrics);
            }
        }

        // Print metrics every 10 seconds
        auto now = std::chrono::steady_clock::now();
        if (std::chrono::duration_cast<std::chrono::seconds>(now - last_metrics_print).count() >= 10) {
            metrics.print();
            last_metrics_print = now;
        }
    }

    // Graceful shutdown
    std::cout << "\n=== Shutting down ===\n";

    for (auto& [fd, conn] : connections) {
        auto duration = std::chrono::steady_clock::now() - conn.connected_at;
        auto seconds = std::chrono::duration_cast<std::chrono::seconds>(duration).count();
// ...continues on next slide
```

epoll() - High-Performance I/O for Linux

Code Examples

📌 6. Production-Grade epoll Server (9/9)

[Code continues across slides]

```
// ...continued from previous slide

        std::cout << "[FD " << fd << "] Closing (duration: " << seconds << "s, "
                  << "rx: " << conn.bytes_received << " bytes, "
                  << "tx: " << conn.bytes_sent << " bytes)\n";

        epoll_ctl(epfd, EPOLL_CTL_DEL, fd, nullptr);
        close(fd);
    }

    close(epfd);
    close(server_fd);

    metrics.print();
    std::cout << "\n=== Server stopped ===\n";

    return 0;
}
```

```
['**Expected Output:**']
```

What You Learned

Key Takeaways

- ✓ The Problem with poll()
- ✓ What is epoll()?
- ✓ How epoll() Works Internally
- ✓ epoll API in Detail

→ Swipe to learn →